

Fault-Module Design (Version 1)

Automated Fault Detection for Reversible Logic Circuits

Tyler Delellis, Simulator and Fault Injection Lead

Scope: this design covers only the three approved fault models: gate removal, control-line modification, and wire swap.

1. Fault Representation

A fault is stored as a Fault object with two fields:

- `fault_type`: "REMOVE", "CONTROL", or "SWAP"
- `location`: a tuple of non-negative integers saying where the fault happens

Gates are numbered by their position in the circuit, starting at 0.

Type	Location	Meaning	Fault ID
REMOVE	$(g,)$	Gate g is removed	<code>REMOVE_G{g}</code>
CONTROL	$(g, \text{old}, \text{new})$	Gate g 's control wire changes from old to new	<code>CONTROL_G{g}_W{old}_TO_W{new}</code>
SWAP	(g, a, b)	Wires a and b are crossed right before gate g	<code>SWAP_G{g}_W{a}_W{b}</code>

Examples (illustrative only):

Fault ID	Meaning
<code>REMOVE_G2</code>	Gate 2 is removed
<code>CONTROL_G1_W0_TO_W2</code>	Gate 1's control moves from wire 0 to wire 2
<code>SWAP_G0_W0_W1</code>	Wires 0 and 1 are crossed before gate 0

2. Injection Rules

All injection works on a copy of the circuit. The original circuit is never changed, so the correct and faulty versions can be compared.

REMOVE: delete gate g from the gate list. This applies to every gate type.

CONTROL: replace one control wire of gate g with a different wire. The new wire must be in range and not already used by that gate. This applies only to gates that have controls:

Gate	Control wires that can be changed
NOT	None (no CONTROL faults)
CNOT	control
TOFFOLI	control1 or control2
FREDKIN	control

SWAP: insert an unconditional swap of wires a and b immediately before gate g . Only pairs with $a < b$ are used, so each pair is counted once. The swap is implemented as a small internal helper step (WireSwap). It is not one of the four project gates and is used only for fault injection.

3. Injection Pseudocode

```

function apply(fault, circuit):
    faulty = deep copy of circuit

    if (fault.fault_type == "REMOVE")
    {
        g = fault.location[0]
        remove faulty.gates[g]
    }
    else if (fault.fault_type == "CONTROL")
    {
        g, old, new = fault.location
        gate = faulty.gates[g]
        build a new gate of the same type, with old replaced by new
            (rebuilding reruns the normal wire validation)
        faulty.gates[g] = new gate
    }
    else if (fault.fault_type == "SWAP")
    {
        g, a, b = fault.location
        insert WireSwap(a, b) into faulty.gates at position g
    }

    faulty.validate()
    return faulty

function get_id(fault):
    if REMOVE: return "REMOVE_G" + g
    if CONTROL: return "CONTROL_G" + g + "_W" + old + "_TO_W" + new
    if SWAP: return "SWAP_G" + g + "_W" + a + "_W" + b

```

4. Generation Pseudocode

FaultGenerator builds every possible fault for one circuit. The order is fixed so repeated runs produce the same list, which keeps the detection-matrix columns stable.

```

function generate_removal_faults(circuit):
    faults = empty list
    for (g = 0; g < number of gates; g++)
    {
        add Fault("REMOVE", (g,))
    }
    return faults

function generate_control_faults(circuit):
    faults = empty list
    for (g = 0; g < number of gates; g++)
    {
        gate = circuit.gates[g]
        for each control wire old of gate (in order)
        {
            for (new = 0; new < num_wires; new++)
            {
                if (new is not used by gate)
                    add Fault("CONTROL", (g, old, new))
            }
        }
    }
    return faults

function generate_swap_faults(circuit):
    faults = empty list

```

```

for (g = 0; g < number of gates; g++)
{
    for (a = 0; a < num_wires; a++)
    {
        for (b = a + 1; b < num_wires; b++)
        {
            add Fault("SWAP", (g, a, b))
        }
    }
}
return faults

function generate_all(circuit):
    return removal faults + control faults + swap faults    (in that order)

```

5. Fault Counts

For a circuit with G gates and n wires:

Fault type	Number of faults
REMOVE	G
CONTROL	For each control wire of each gate: n minus the number of wires that gate uses
SWAP	$G \times n \times (n - 1) / 2$

Example only: a 2-wire circuit with one CNOT(0, 1).

Fault type	Count	Fault IDs
REMOVE	1	REMOVE_G0
CONTROL	0	None (both wires are already used by the gate)
SWAP	1	SWAP_G0_W0_W1
Total	2	

6. Notes

- Some faults may produce the same outputs as the correct circuit, for example crossing the two controls of a Toffoli gate. These faults are undetectable by any test vector and are reported as such in the detection matrix rather than removed.
- Fault IDs are used as the detection-matrix column labels.

7. Open Items for Team Confirmation

- Wire-swap location: the crossing happens right before gate g , so the swap fault ID includes the gate number (SWAP_G{ g }_W{ a }_W{ b }). This replaces the earlier SWAP_W{ a }_W{ b } format in simulator_output_format.md.
- Whether a swap after the last gate should also be included.